

Using MLP for Non-linear Regression

1. Introduction

In this project I used a MLP with non linear functions and a Mean Squared Error function to fit a line to points on a graph.

2.Method

The training loop uses an MSE function from pytorch and collects the loss data to be graphed later

```
for epoch in range(num_epochs):

    model = model.train()
    epoch_loss = 0
    for features, labels in train_loader:

        logits = model(features)
        labels = labels.view(-1,1)
        loss = F.mse_loss(logits, labels) # Loss function

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    avg_epoch_loss = epoch_loss / len(train_loader)
    training_losses.append(avg_epoch_loss)
    train_acc = compute_accuracy(model, train_loader)
    print(f"Epoch {epoch+1}/{num_epochs}, Train ACC: {train_acc:.4f}")
```

The MLP has 3 hidden layers that use rectifier activation functions

```
self.all_layers = torch.nn.Sequential(  
    # 1st hidden layer  
    torch.nn.Linear(num_features, 500),  
    torch.nn.ReLU(),  
  
    # 2nd hidden layer  
    torch.nn.Linear(500, 300),  
    torch.nn.ReLU(),  
    # 3rd hidden layer  
    torch.nn.Linear(300, 100),  
    torch.nn.ReLU(),  
  
    # output layer  
    torch.nn.Linear(100, 1),  
)
```

The base hyper parameters where:

batch_size=200

Learning rate= 0.001

epochs = 500

And the MLP shown above

=====

The second test hyper parameters are:

batch_size=64

Learning rate= 0.001(same as before)

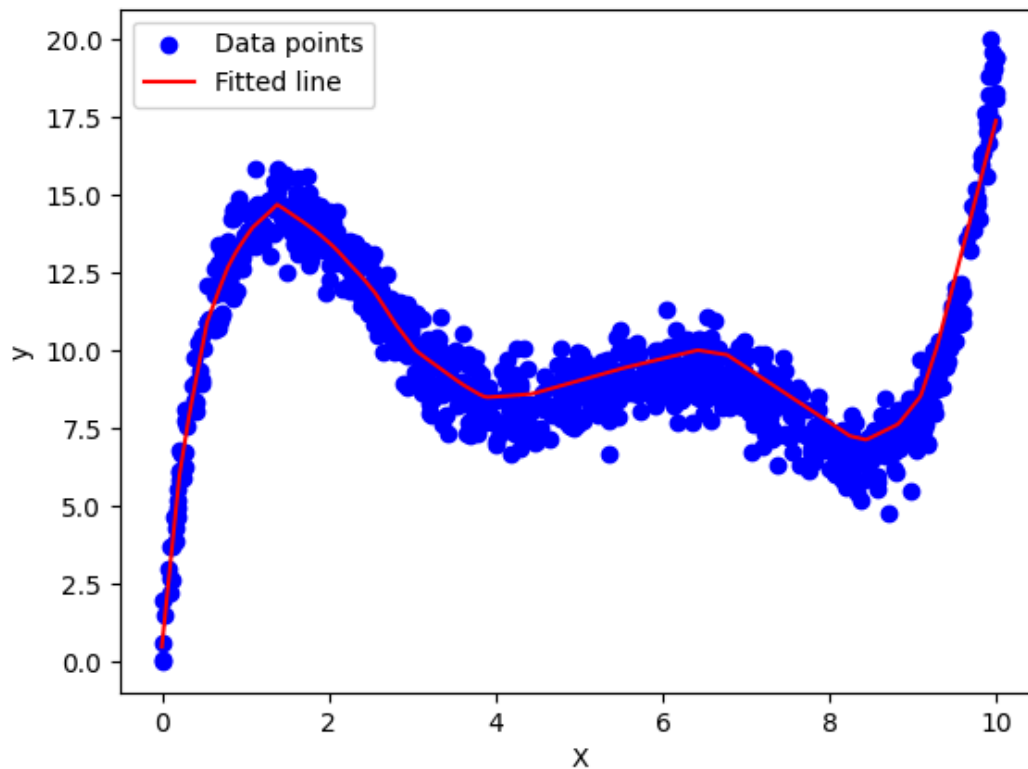
epochs = 500(same as before)

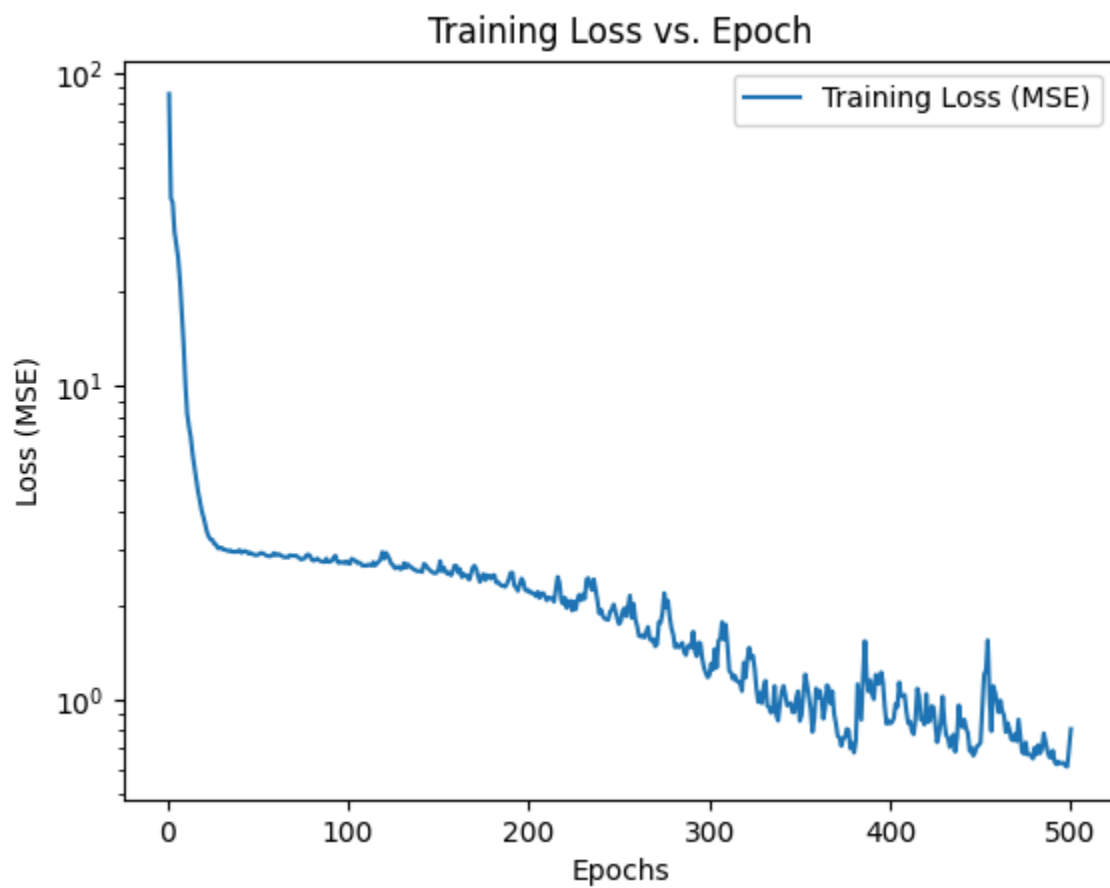
And the MLP shown below (less neurons per layer)

```
self.all_layers = torch.nn.Sequential(  
    # 1st hidden layer  
    torch.nn.Linear(num_features, 5),  
    torch.nn.ReLU(),  
    # 2nd hidden layer  
    torch.nn.Linear(5, 64),  
    torch.nn.ReLU(),  
    # 3rd hidden layer  
    torch.nn.Linear(64, 20),  
    torch.nn.ReLU(),  
    # output layer  
    torch.nn.Linear(20, 1),  
)
```

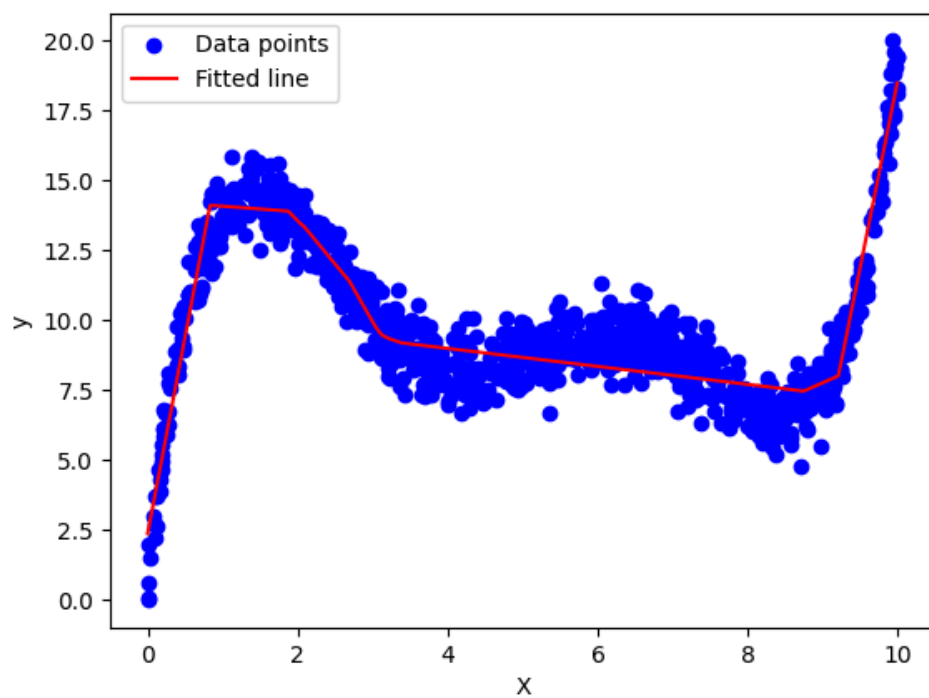
3.Results

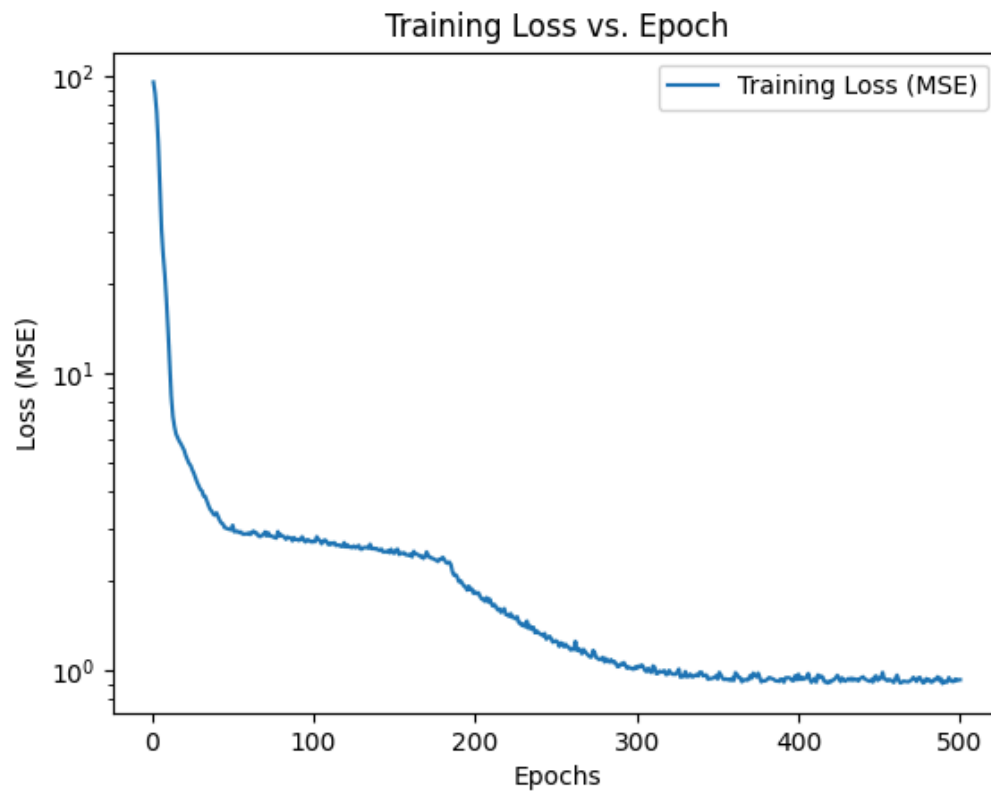
These are the base results





These are the second results:





4. Analysis

The base results were superior to the second results. The two main changes were the MLP and the `batch_size`. In a previous test I kept the base hyper parameters and only changed one of those two parameters. I found that the big reduction in neurons per layer causes a much greater drop in quality than lowering the batch size. The loss graphs between the two show that the base approach was much less stable near the end of the epochs.

5. Conclusion

When building a MLP the hyper parameters you choose can make a large difference in the results you can achieve. In this small simple model I was able to use a large amount of neurons per layer, but in more complex models the performance costs might be too high to use that many neurons.